



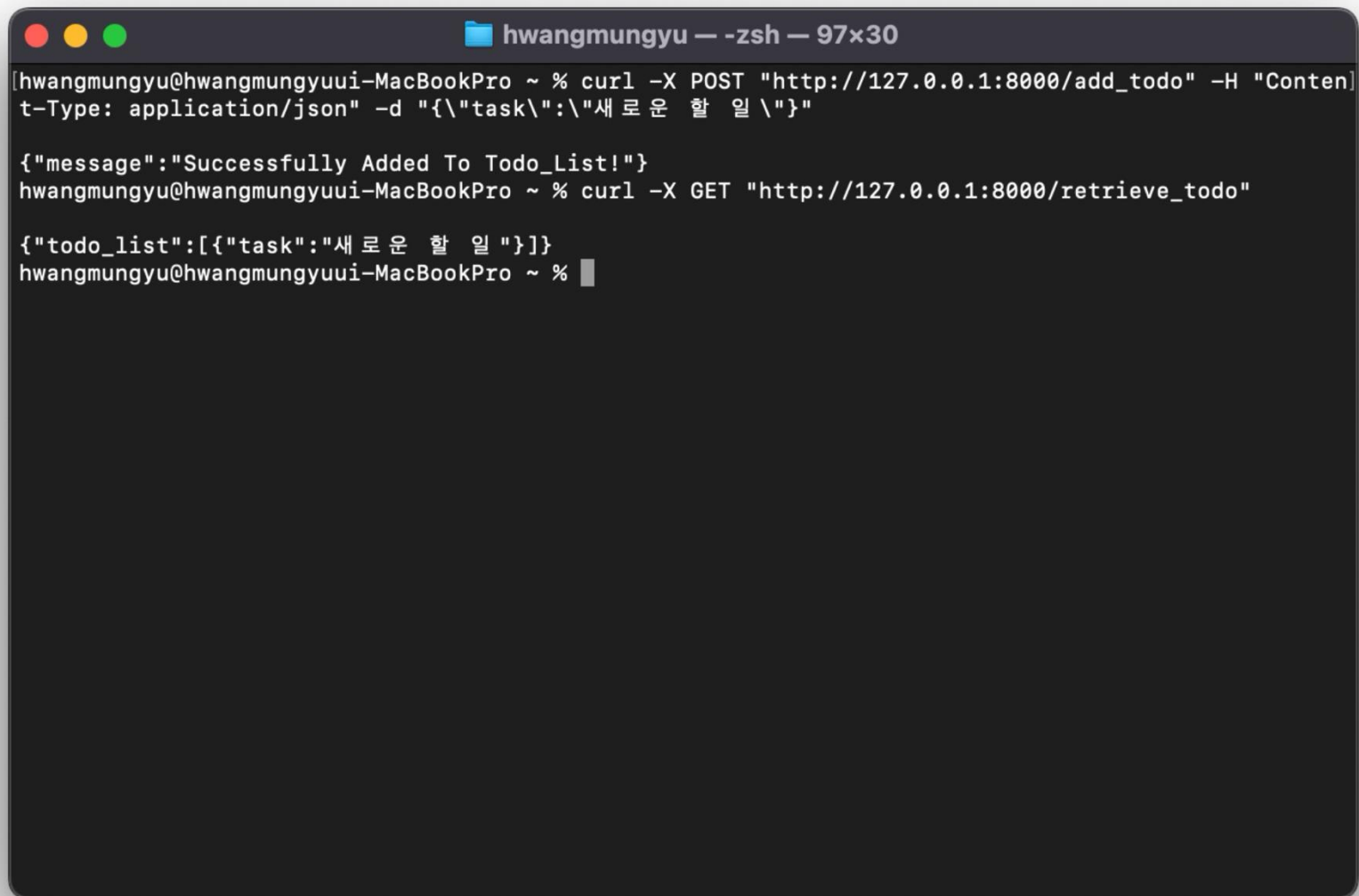
SW Pilot 학습보고서 <과정 7>

파이썬 5팀 황문규

문제 1. 또 새로운 프로젝트 - 코드

```
1 from fastapi import FastAPI, APIRouter, HTTPException, Request
2 from typing import Dict, List
3
4 app = FastAPI()
5 router = APIRouter()
6
7 todo_list: List[Dict] = []
8
9 @router.post("/add_todo")
10 async def add_todo(request: Request):
11     data = await request.json()
12     if not data or 'task' not in data or not data['task']:          # 보너스 과제 - 입력되는 Dict 타입이 빈값이면 경고
13         raise HTTPException(status_code=400, detail="Task cannot be empty")
14     todo_list.append(data)
15     return {"message": "Successfully Added To Todo_List!"}
16
17 @router.get("/retrieve_todo")
18 async def retrieve_todo():
19     return {"todo_list": todo_list}
20
21 app.include_router(router)
```

문제 1. 또 새로운 프로젝트 - 수행화면



```
hwangmungyu — -zsh — 97x30
[hwangmungyu@hwangmungyuui-MacBookPro ~ % curl -X POST "http://127.0.0.1:8000/add_todo" -H "Content-Type: application/json" -d '{"task":"새로운 할 일"}'

{"message":"Successfully Added To Todo_List!"}
hwangmungyu@hwangmungyuui-MacBookPro ~ % curl -X GET "http://127.0.0.1:8000/retrieve_todo"

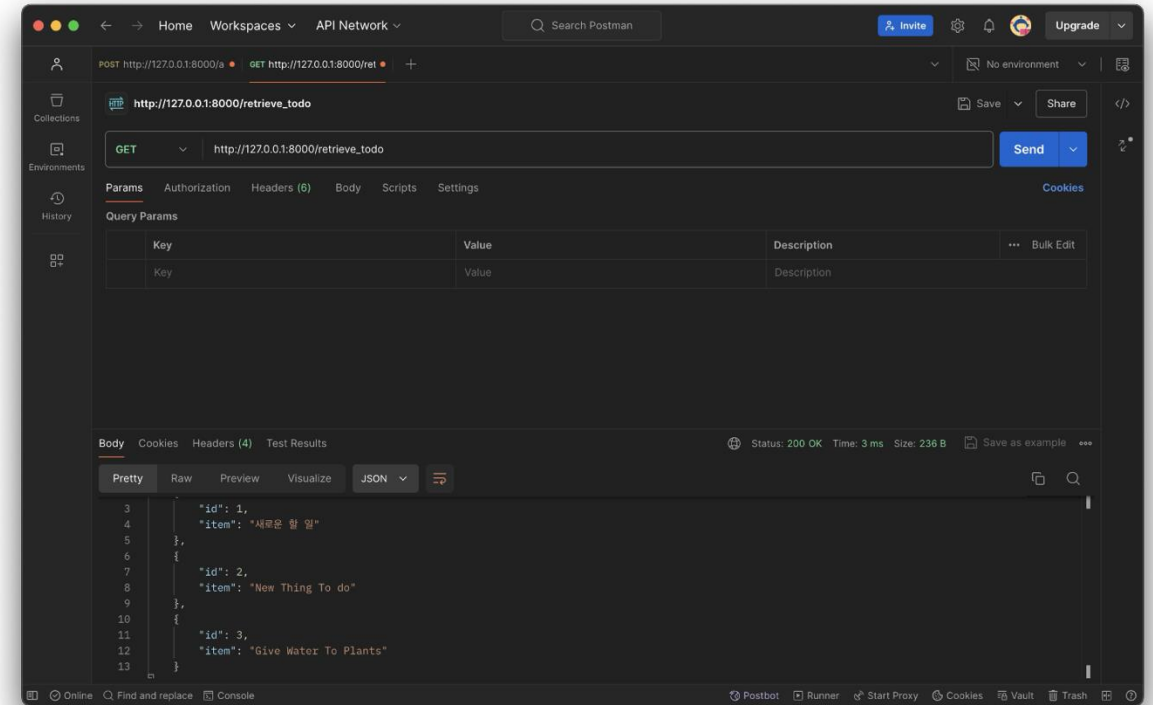
{"todo_list":[{"task":"새로운 할 일"}]}
hwangmungyu@hwangmungyuui-MacBookPro ~ %
```

문제 2. 정확한 To-Do - 코드

```
1 from fastapi import FastAPI, APIRouter, HTTPException
2 from pydantic import BaseModel
3 from typing import List
4
5 app = FastAPI()
6 router = APIRouter()
7
8 todo_list: List[BaseModel] = []
9
10 class Todo(BaseModel):
11     id: int
12     item: str
13
14 @router.post("/add_todo", response_model=Todo)
15 async def add_todo(todo: Todo):
16     if not todo.item:
17         raise HTTPException(status_code=400, detail="Item cannot be empty")
18     todo_list.append(todo)
19     return todo
20
21 @router.get("/retrieve_todo", response_model=List[Todo])
22 async def retrieve_todo():
23     return todo_list
24
25 app.include_router(router)
```

문제 2. 정확한 To-Do - 수행화면

```
hwangmungyu@hwangmungyuui-MacBookPro ~ % curl -X POST "http://127.0.0.1:8000/add_todo" -H "Content-Type: application/json" -d '{"id" : 1, "item" : ""}'  
{"detail": "Item cannot be empty"}  
hwangmungyu@hwangmungyuui-MacBookPro ~ %
```



문제 3. 완전히 작동하는 Todo - 코드



```
1 from pydantic import BaseModel
2
3 class Todo(BaseModel):
4     id: int
5     item: str
6
7 class TodoItem(BaseModel):
8     item: str
```



```
1 from fastapi import FastAPI, APIRouter, HTTPException
2 from typing import List
3 from model import Todo, TodoItem # model.py에서 모델을 가져옴
4
5 app = FastAPI()
6 router = APIRouter()
7
8 todo_list: List[Todo] = []
9
10 # 과제 1, 2에서 완성한 기능
11 @router.post("/add_todo", response_model=Todo)
12 async def add_todo(todo: Todo):
13     if not todo.item:
14         raise HTTPException(status_code=400, detail="Item cannot be empty")
15     todo_list.append(todo)
16     return todo
17
18 @router.get("/retrieve_todo", response_model=List[Todo])
19 async def retrieve_todo():
20     return todo_list
21
22 # 과제 3 구현 가능
23 @router.get("/get_single_todo/{todo_id}", response_model=Todo)
24 async def get_single_todo(todo_id: int):
25     for todo in todo_list:
26         if todo.id == todo_id:
27             return todo
28     raise HTTPException(status_code=404, detail="Todo not found")
29
30 @router.put("/update_todo/{todo_id}", response_model=Todo)
31 async def update_todo(todo_id: int, updated_todo: TodoItem):
32     print(f"Received update request for todo_id: {todo_id} with data: {updated_todo}")
33     for todo in todo_list:
34         if todo.id == todo_id:
35             todo.item = updated_todo.item
36             print(f"Updated todo: {todo}")
37             return todo
38     print("Todo not found")
39     raise HTTPException(status_code=404, detail="Todo not found")
40
41 @router.delete("/delete_single_todo/{todo_id}")
42 async def delete_single_todo(todo_id: int):
43     for todo in todo_list:
44         if todo.id == todo_id:
45             todo_list.remove(todo)
46             return {"message": "Todo deleted successfully"}
47     raise HTTPException(status_code=404, detail="Todo not found")
48
49 app.include_router(router)
```

문제 3. 완전히 작동하는 Todo - 코드 (보너스 과제)

```
1 # 보너스 과제 - 구현한 기능들을 호출해서 동작하는 간단한 클라이언트 앱
2 import requests
3
4 BASE_URL = "http://127.0.0.1:8000"
5
6 def add_todo(ID, item):
7     response = requests.post(f"{BASE_URL}/add_todo", json={"id": ID, "item": item})
8     if response.status_code == 200:
9         print("Todo added:", response.json())
10    else:
11        print("Failed to add todo:", response.json())
12
13 def retrieve_todos():
14     response = requests.get(f"{BASE_URL}/retrieve_todo")
15     if response.status_code == 200:
16         print("Todo list:", response.json())
17     else:
18         print("Failed to retrieve todos:", response.json())
19
20 def get_single_todo(ID):
21     response = requests.get(f"{BASE_URL}/get_single_todo/{ID}")
22     if response.status_code == 200:
23         print("Todo retrieved:", response.json())
24     else:
25         print("Failed to retrieve todo:", response.json())
26
27 def update_todo(ID, item):
28     response = requests.put(f"{BASE_URL}/update_todo/{ID}", json={"item": item})
29     if response.status_code == 200:
30         print("Todo updated:", response.json())
31     else:
32         try:
33             print("Failed to update todo:", response.json())
34         except requests.exceptions.JSONDecodeError:
35             print("Failed to update todo: No JSON response received. Status code:", response.status_code)
36
```

```
1 def delete_todo(ID):
2     response = requests.delete(f"{BASE_URL}/delete_single_todo/{ID}")
3     if response.status_code == 200:
4         print("Todo deleted:", response.json())
5     else:
6         print("Failed to delete todo:", response.json())
7
8 if __name__ == "__main__":
9     # Test
10    print("Adding todos:")
11    add_todo(1, "Give Water To Plants")
12    add_todo(2, "Communicate With Earth")
13
14    print("\nRetrieving all todos:")
15    retrieve_todos()
16
17    print("\nGetting single todo (ID 1):")
18    get_single_todo(1)
19
20    print("\nUpdating todo (ID 1):")
21    update_todo(1, "Give Water To Groceries")
22
23    print("\nGetting updated single todo (ID 1):")
24    get_single_todo(1)
25
26    print("\nDeleting todo (ID 1):")
27    delete_todo(1)
28
29    print("\nRetrieving all todos after deletion:")
30    retrieve_todos()
```

문제 3. 완전히 작동하는 Todo - 수행화면

```
Level3 - -zsh - 109x36
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Level3 % python3 clientApp.py
Adding todos:
Todo added: {'id': 1, 'item': 'Give Water To Plants'}
Todo added: {'id': 2, 'item': 'Communicate With Earth'}

Retrieving all todos:
Todo list: [{'id': 1, 'item': 'Give Water To Plants'}, {'id': 2, 'item': 'Communicate With Earth'}]

Getting single todo (ID 1):
Todo retrieved: {'id': 1, 'item': 'Give Water To Plants'}

Updating todo (ID 1):
Todo updated: {'id': 1, 'item': 'Give Water To Groceries'}

Getting updated single todo (ID 1):
Todo retrieved: {'id': 1, 'item': 'Give Water To Groceries'}

Deleting todo (ID 1):
Todo deleted: {'message': 'Todo deleted successfully'}

Retrieving all todos after deletion:
Todo list: [{'id': 2, 'item': 'Communicate With Earth'}]
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Level3 %
```


문제 4. 알 수 없는 오류 - 코드

```
1 from fastapi import FastAPI, APIRouter, HTTPException, status
2 from typing import List
3 from model import Todo, TodoItem, TodoResponse # model.py에서 모델을 가져옴
4
5 app = FastAPI()
6 router = APIRouter()
7
8 todo_list: List[Todo] = []
9
10 # 새로운 Todo 항목을 추가하는 엔드포인트
11 @router.post("/add_todo", response_model=Todo, status_code=status.HTTP_201_CREATED)
12 async def add_todo(todo: Todo):
13     # 예외 상황: 항목이 비어 있을 때 (400 Bad Request)
14     if not todo.item:
15         raise HTTPException(
16             status_code=status.HTTP_400_BAD_REQUEST,
17             detail="Item cannot be empty"
18         )
19     todo_list.append(todo)
20     return todo
21
22 # 모든 Todo 항목을 조회하는 엔드포인트
23 @router.get("/retrieve_todo", response_model=TodoResponse, status_code=status.HTTP_201_CREATED)
24 async def retrieve_todo():
25     # 예외 상황: Todo 목록이 비어 있을 때 (404 Not Found)
26     if not todo_list:
27         raise HTTPException(
28             status_code=status.HTTP_404_NOT_FOUND,
29             detail="No todos found"
30         )
31
32     # 보너스 과제 - 응답 모델을 사용하는 방식으로 수정
33     return TodoResponse(todos=todo_list, message="Todos retrieved successfully")
34
35 # 특정 Todo 항목을 조회하는 엔드포인트
36 @router.get("/get_single_todo/{todo_id}", response_model=Todo, status_code=status.HTTP_201_CREATED)
37 async def get_single_todo(todo_id: int):
38     for todo in todo_list:
39         if todo.id == todo_id:
40             return todo
41
42     # 예외 상황: 요청한 ID의 Todo 항목이 없을 때 (404 Not Found)
43     raise HTTPException(
44         status_code=status.HTTP_404_NOT_FOUND,
45         detail="Todo not found"
46     )
47
```

```
1 # 특정 Todo 항목을 업데이트하는 엔드포인트
2 @router.put("/update_todo/{todo_id}", response_model=Todo, status_code=status.HTTP_201_CREATED)
3 async def update_todo(todo_id: int, updated_todo: TodoItem):
4     for todo in todo_list:
5         if todo.id == todo_id:
6             # 예외 상황: 업데이트할 항목이 비어 있을 때 (400 Bad Request)
7             if not updated_todo.item:
8                 raise HTTPException(
9                     status_code=status.HTTP_400_BAD_REQUEST,
10                     detail="Updated item cannot be empty"
11                 )
12
13             todo.item = updated_todo.item
14             return todo
15
16     # 예외 상황: 요청한 ID의 Todo 항목이 없을 때 (404 Not Found)
17     raise HTTPException(
18         status_code=status.HTTP_404_NOT_FOUND,
19         detail="Todo not found"
20     )
21
22 # 특정 Todo 항목을 삭제하는 엔드포인트
23 @router.delete("/delete_single_todo/{todo_id}", status_code=status.HTTP_201_CREATED)
24 async def delete_single_todo(todo_id: int):
25     for todo in todo_list:
26         if todo.id == todo_id:
27             todo_list.remove(todo)
28             return {"message": "Todo deleted successfully"}
29
30     # 예외 상황: 요청한 ID의 Todo 항목이 없을 때 (404 Not Found)
31     raise HTTPException(
32         status_code=status.HTTP_404_NOT_FOUND,
33         detail="Todo not found"
34     )
35
36 app.include_router(router)
37
```

문제 4. 알 수 없는 오류 - 수행화면

```
Level3 — -zsh — 109x36
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Level3 % curl -X GET "http://127.0.0.1:8000/retrieve_todo"
{"detail":"No todos found"}%
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Level3 % curl -X GET "http://127.0.0.1:8000/get_single_todo/999"
{"detail":"Todo not found"}%
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Level3 % curl -X PUT "http://127.0.0.1:8000/update_todo/999" -H
"Content-Type: application/json" -d '{"item": "Updated Todo"}'
{"detail":"Todo not found"}%
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Level3 % curl -X DELETE "http://127.0.0.1:8000/delete_single_todo/999"
{"detail":"Todo not found"}%
(myenv) hwangmungyu@hwangmungyuui-MacBookPro Level3 %
```